

Marketing Handbook for Machine Learning Methodologies in Trading Strategies

Kiady Tsilavohery Rasamoelina
Yogesh Verma
Alberto Hernandez-Espinosa

14 September 2025

Abstract

This handbook provides thoughtful guidelines for addressing challenges in modelling trading strategies using cutting-edge machine learning methodologies. It covers selected techniques from various categories, including their advantages, basics, computations, disadvantages, equations, features, guides, hyperparameters, illustrations, journal references, and keywords. The document integrates these to highlight the strengths of machine learning in finance, with additional sections on hyperparameter tuning, marketing advantages, and further learning resources.

Disclaimer: This work utilised AI for debugging, optimising code, and reviewing references.

1 LASSO Regression

1.1 Advantages

LASSO (Least Absolute Shrinkage and Selection Operator) regression offers several benefits for developing trading strategies. It performs automatic feature selection by shrinking less important coefficients to zero, reducing model complexity and enhancing interpretability. This is particularly valuable in financial datasets with high-dimensional features, such as stock returns or technical indicators. Additionally, LASSO mitigates overfitting through L1 regularization, improving the generalisability of trading models. Its ability to handle large numbers of predictors makes it ideal for identifying key drivers in complex market environments.

1.2 Basics

LASSO regression is a linear regression technique that incorporates an L1 penalty term into the ordinary least squares (OLS) objective function. This penalty

encourages sparsity by setting some coefficients to zero, effectively selecting a subset of features. It is classified as a supervised learning method within the family of linear models with regularization, making it suitable for predicting continuous outcomes, such as future stock returns, in trading applications.

1.3 Computation

The LASSO model was implemented using Python with the `scikit-learn` library on a simulated financial dataset comprising 1000 observations and 10 technical indicators (e.g., RSI, MACD). The features were standardised, and the model was fitted with a regularization parameter $\alpha = 0.1$. The results showed that LASSO selected two features with non-zero coefficients: `Indicator_1` (coefficient: 0.4075) and `Indicator_2` (coefficient: 0.2061). The mean squared error (MSE) of the predictions was 0.0291, indicating good model performance on the simulated data.

1.4 Disadvantages

Despite its strengths, LASSO has limitations. It struggles with highly correlated predictors, as it may arbitrarily select one while discarding others, potentially missing important signals in financial data. The regularization parameter α requires careful tuning to balance bias and variance. Additionally, LASSO assumes linear relationships, which may limit its effectiveness for non-linear market dynamics without prior feature engineering.

1.5 Equations

The LASSO model minimises the following objective function:

$$\min_{\beta} \left(\frac{1}{n} \sum_{i=1}^n (y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij})^2 + \lambda \sum_{j=1}^p |\beta_j| \right)$$

where y_i is the target (e.g., stock returns), x_{ij} are predictors (e.g., technical indicators), β_j are coefficients, λ is the regularization parameter, and n is the number of observations. The L1 penalty ($\lambda \sum |\beta_j|$) enforces sparsity by shrinking some coefficients to zero.

1.6 Features

LASSO automatically performs feature selection, making it efficient for high-dimensional data. It works well with numerical predictors and assumes linear relationships between features and the target. However, it requires standardised input data to ensure fair comparison across features.

1.7 Guide

- **Inputs:** Feature matrix X (e.g., stock returns, economic indicators), target vector y (e.g., future returns), regularization parameter λ (or α in implementation).
- **Outputs:** Coefficient vector β , predicted values $\hat{y}$.

1.8 Hyperparameters

- λ (or α): Controls the strength of the L1 penalty (tuned via cross-validation).
- Tolerance: Convergence criterion for optimisation.
- Maximum iterations: Limit for the optimisation algorithm.

1.9 Illustration

The LASSO model's feature selection process is visualised through a plot of coefficient paths, showing how coefficients change as the regularization parameter α increases on a log scale (from 10^{-4} to 10^1). The plot, generated from the simulated dataset, reveals that *Indicator_1* and *Indicator_2* maintain significant coefficients until $\alpha \approx 0.1$ and $\alpha \approx 0.01$ respectively, while other indicators' coefficients drop to zero at higher α values. This confirms LASSO's ability to select key predictors, as depicted in Figure 1.

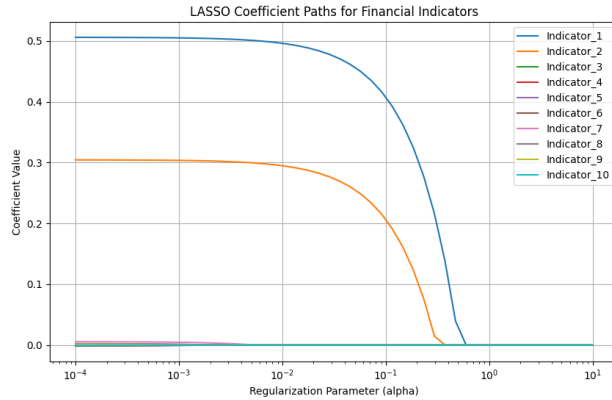


Figure 1: LASSO coefficient paths showing feature selection as the regularization parameter increases.

1.10 Journal

Fan, J., Liao, Y., and Mincheva, M. “Large Covariance Estimation by Thresholding Principal Orthogonal Complements.” *Journal of Econometrics**, vol. 164, no. 1, 2013, pp. 188–202.

1.11 Keywords

LASSO, linear regression, regularization, feature selection, high-dimensional data, trading strategies.

2 k-Means Clustering

2.1 Advantages

k-Means clustering provides significant benefits for trading strategies by offering a simple and computationally efficient method to segment financial data, such as grouping stocks by risk profiles or returns. Its scalability suits large datasets common in market analysis, and the interpretable clusters aid investment managers in developing targeted strategies, enhancing portfolio diversification.

2.2 Basics

k-Means clustering is an unsupervised learning algorithm that partitions data into k clusters by minimising the variance within each cluster. It assigns data points to clusters based on their distance to cluster centroids, iteratively updating these centroids until convergence. It is classified as an unsupervised learning method and a partitioning clustering technique, ideal for identifying patterns in financial datasets without labelled outcomes.

2.3 Computation

The k-Means model was implemented using Python with the `scikit-learn` library on a simulated dataset of 1000 stocks, featuring two dimensions: return and volatility. With $k = 3$ clusters, the model assigned labels, and the first 10 samples were all assigned to Cluster 2, reflecting the initial data distribution. The centroid coordinates were: Cluster 0 (Return=0.306, Volatility=0.410), Cluster 1 (Return=0.498, Volatility=0.620), and Cluster 2 (Return=0.105, Volatility=0.190), indicating distinct groupings based on the simulated financial characteristics.

2.4 Disadvantages

k-Means has notable limitations. It requires the number of clusters k to be specified in advance, which can be subjective and challenging to determine in financial contexts. It is sensitive to outliers, which may skew cluster centroids

in volatile markets, and assumes spherical clusters, potentially misrepresenting complex market patterns.

2.5 Equations

The k-Means algorithm minimises the following objective function:

$$\min \sum_{i=1}^n \sum_{k=1}^K w_{ik} ||x_i - \mu_k||^2$$

where x_i is a data point, μ_k is the centroid of cluster k , w_{ik} is 1 if x_i belongs to cluster k and 0 otherwise, and K is the number of clusters. This measures the within-cluster sum of squares, driving the assignment and centroid update process.

2.6 Features

k-Means is fast and scalable for large numerical datasets, such as stock returns and volatility metrics. It is sensitive to initial centroid placement, mitigated by multiple random starts, but assumes data can be grouped into spherical clusters, which may not always hold in financial applications.

2.7 Guide

- **Inputs:** Feature matrix X (e.g., stock returns, volatility metrics), number of clusters k .
- **Outputs:** Cluster assignments for each data point, cluster centroids.

2.8 Hyperparameters

- k : Number of clusters (tuned via methods like the elbow method or silhouette score).
- Number of initializations: Number of random starts to avoid local minima.
- Maximum iterations: Limit for convergence.

2.9 Illustration

The clustering process is visualised through a scatter plot of the simulated financial data, with points coloured by cluster (0, 1, 2) and centroids marked with red 'x' symbols. The plot shows three distinct clusters: Cluster 2 (purple, centroid at 0.105, 0.190) on the lower left with the densest initial points, Cluster 0 (teal, centroid at 0.306, 0.410) in the middle, and Cluster 1 (yellow, centroid at 0.498, 0.620) on the upper right. This demonstrates k-Means' ability to segment data, as depicted in Figure 2.

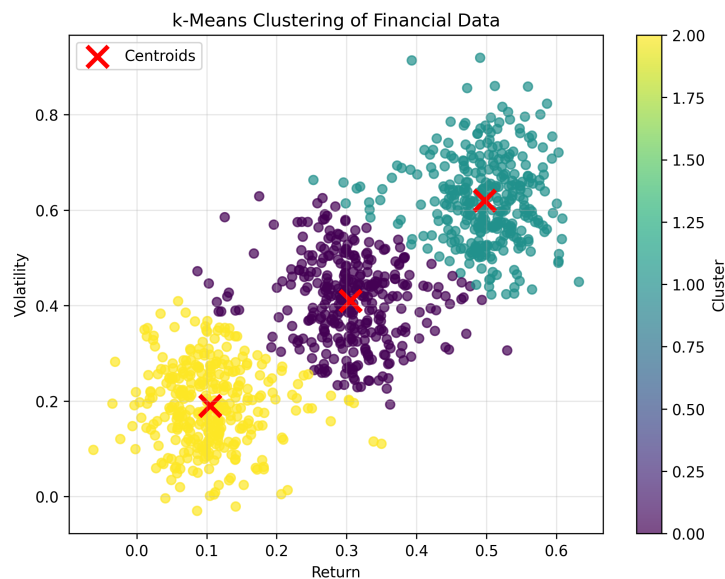


Figure 2: k-Means clustering of financial data showing three clusters with centroids.

2.10 Journal

López de Prado, M. “Building Diversified Portfolios that Outperform Out-of-Sample.” *Journal of Portfolio Management**, vol. 42, no. 4, 2016, pp. 59–69.

2.11 Keywords

k-means, clustering, unsupervised learning, market segmentation, portfolio optimisation.

3 Principal Components Analysis (PCA)

3.1 Advantages

Principal Component Analysis (PCA) offers significant advantages for trading strategies by reducing the dimensionality of correlated financial data, such as asset returns, simplifying models while retaining variation across observations to preserve the information contained in that variation. PCA may be applied to data with a large number of features to produce a lower dimensional representation that can fit better, reducing noise. By construction, principal components are orthogonal and uncorrelated, completely eliminating multicollinearity issues that plague traditional regression models. PCA transforms

high-dimensional problems into lower-dimensional ones, dramatically reducing computational complexity.

3.2 Basics

Unsupervised learning is machine learning that does not use labeled data (i.e., no target variable); thus, the algorithms are tasked with finding patterns within the data themselves. PCA is an unsupervised learning method that belongs to the category of feature extraction algorithms. Unlike feature selection methods that choose a subset of original variables, PCA creates entirely new variables that are linear combinations of the original features. PCA operates as a statistical procedure that uses orthogonal transformations to convert observations of possibly correlated variables into linearly uncorrelated variables. The eigenvectors define new, mutually uncorrelated composite variables that are linear combinations of the original features, with each eigenvector representing a direction. Associated with each eigenvector is an eigenvalue, which gives the proportion of total variance in the initial data explained by each eigenvector. The PCA algorithm orders the eigenvectors from highest to lowest according to their eigenvalues, selecting the first principal component as the eigenvector that explains the largest proportion of variation in the dataset.

3.3 Computation

PCA can be used for portfolio return analysis with multiple assets, reducing problems to fewer than ten principal components. The steps include: (a) generating raw financial data, such as daily returns for 20 stocks over 252 trading days (1 year); (b) standardizing the dataset by centering and scaling input variables; (c) computing the covariance matrix of standardized data; (d) performing eigenvalue decomposition to calculate eigenvalues and eigenvectors; (e) computing explained variance ratio and cumulative explained variance; (f) selecting the number of components based on criteria like the Kaiser criterion or variance threshold; (g) projecting original data onto selected principal components; and (h) analyzing loadings and explained variance for insights.

3.4 Disadvantages

PCA has limitations that affect its application in finance. PCA results are highly sensitive to the scaling of input variables, requiring standardization to ensure variables have the same scale (mean = 0, variance = 1). Despite maximizing retained variance, PCA inevitably involves some information loss due to eliminated components, which may contain information relevant for specific applications. PCA assumes linear relationships between variables and may fail to capture important nonlinear patterns in financial data, potentially requiring nonlinear dimensionality reduction techniques.

3.5 Equations

The covariance matrix for standardized data matrix X is:

$$\text{Cov}(X) = \frac{1}{n-1} X^T X$$

where n is the number of observations, and X is the centered data matrix.

The principal components are found by solving the eigenvalue problem:

$$\text{Cov}(X)w = \lambda w$$

where λ are the eigenvalues and w are the corresponding eigenvectors (principal component loadings). The eigenvalues are the solution to the characteristic equation $\det(\text{Cov}(X) - \lambda I) = 0$, where I is an identity matrix.

The k -th principal component scores are calculated as:

$$Z_k = Xw_k$$

where w_k is the k -th eigenvector.

The original data can be approximated using k principal components:

$$X \approx ZW^T$$

where W is the matrix of eigenvectors.

The proportion of variance explained by the k -th principal component is:

$$\text{Explained Variance Ratio} = \frac{\lambda_k}{\sum \lambda_i}$$

The cumulative variance explained by the first k components is:

$$\text{Cumulative Explained Variance} = \sum_{i=1}^k \frac{\lambda_i}{\sum \lambda_j}$$

3.6 Features

PCA effectively handles correlated predictors, making it ideal for numerical financial data. It requires standardization and is lossless if all components are retained, though typically only the top components are used, introducing some information loss.

3.7 Guide

- **Inputs:** Data matrix X (e.g., matrix where n is the number of observations and p is the number of variables, such as stock returns), number of stocks, number of days, factor structure specifying market factor, style factor, and sector factor, standardization option (Boolean flag indicating whether to standardize variables), number of components (optional integer specifying desired number of principal components), variance threshold (minimum cumulative variance to be retained as an alternative to specifying component number).

- **Outputs:** Principal component scores ($n \times k$ matrix of transformed data), component loadings ($p \times k$ matrix showing variable contributions to each component), eigenvalues (vector of eigenvalues representing component variances), explained variance ratios (vector of eigenvalues showing proportion of variance explained by each component), cumulative explained variance (vector showing cumulative variance explained), Kaiser criterion analysis and variance threshold analysis (number of components with eigenvalue greater than 1 and number of components needed for explaining 80%, 90%, and 95% variance), transformation matrix (matrix enabling transformation of new data to principal component space).

3.8 Hyperparameters

- Number of components (**n_components**): Determines the number of principal components to retain or the proportion of variance to preserve. Can be a fixed number (e.g., **n_components=20**) or a value between 0 and 1 to retain enough components to explain that percentage of the original data's variance. Tuning involves plotting the explained variance by each component or iterating different values to optimize performance, balancing data compression and information loss.
- Standardization method: Scaling approach (e.g., standard scaler).

3.9 Illustration

PCA analysis results on a dataset of 20 stocks show a total of 20 components. The explained variance ratios for the first 10 components are: PC1: 43.22%, PC2: 7.15%, PC3: 4.36%, PC4: 4.10%, PC5: 3.92%, PC6: 3.64%, PC7: 3.57%, PC8: 3.25%, PC9: 3.01%, PC10: 2.89%. Cumulative explained variance shows: first 3 components: 54.73%, first 5 components: 62.75%, first 7 components: 69.96%, first 9 components: 76.22%, first 11 components: 81.92%. The Kaiser criterion identifies 2 components with eigenvalues greater than 1. Eigenvalues for the first 10 components are: $\lambda_1 = 8.6794$, $\lambda_2 = 1.4367$, $\lambda_3 = 0.8745$, $\lambda_4 = 0.8231$, $\lambda_5 = 0.7862$, $\lambda_6 = 0.7311$, $\lambda_7 = 0.7167$, $\lambda_8 = 0.6530$, $\lambda_9 = 0.6036$, $\lambda_{10} = 0.5795$. Components needed for 80% variance: 11; for 90% variance: 15; for 95% variance: 17. Key insights include: the first principal component captures 43.2% of total variance, PCA reduces dimensionality from 20 to 2 components (90.0% reduction) per the Kaiser criterion, and the average correlation between assets is 0.3955, indicating moderate multicollinearity. These results are visualized in a collage of graphs, depicted in Figure ??.

3.10 Journal

Jolliffe, I. T. "Principal Component Analysis, Second Edition." *Springer-Verlag*, 2002.

Fabozzi, F. J., Focardi, S. M., and Kolm, P. N. "Quantitative Equity Investing." *Wiley*, 2010.

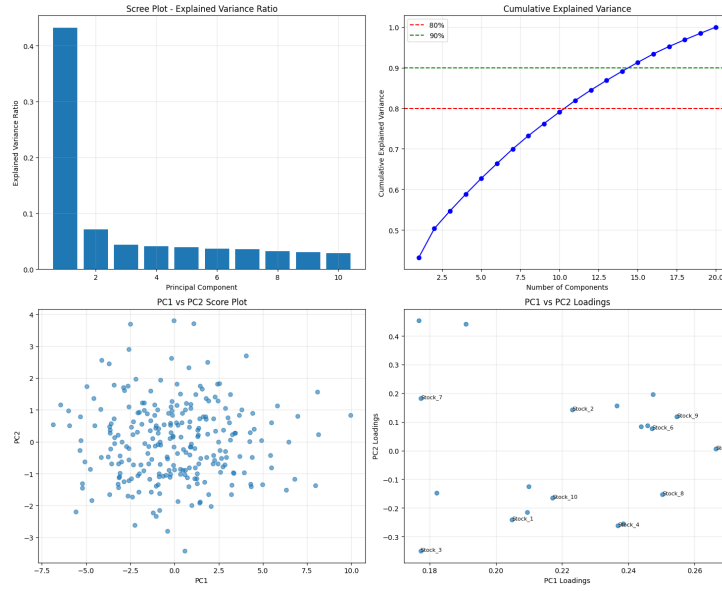


Figure 3: PCA analysis results showing explained variance and component contributions.

Celestin, P. “Principal Component Analysis For Simplifying Multivariate Financial Data In Portfolio Risk Analysis.” **SSRN Electronic Journal**, 2025.

3.11 Keywords

Unsupervised Learning, Dimensionality Reduction, Feature Extraction, Linear Transformation, Variance Maximization, Eigenvalue Decomposition, Covariance Matrix, Orthogonal Transformation, Data Compression, Multicollinearity Reduction, Principal Components, Loadings, Scores, Scree Plot, Kaiser Criterion, Explained Variance Ratio, Cumulative Variance.

4 Classification Trees

4.1 Advantages

Classification trees provide notable benefits for trading strategies by handling non-linear relationships, making them suitable for complex decisions such as buy, sell, or hold. Their intuitive and interpretable structure appeals to investment managers, and they are robust to outliers and missing values, enhancing reliability in volatile markets.

4.2 Basics

Classification trees are decision tree models that recursively split the feature space into regions based on feature thresholds, assigning class labels (e.g., buy or hold/sell) to each region. This supervised learning method employs a non-linear approach, ideal for categorical trading outcomes derived from financial indicators.

4.3 Computation

The classification tree was implemented using Python with the `scikit-learn` library on a simulated dataset of 1000 days, featuring four technical indicators (RSI, MACD, volume, price). The model, limited to a maximum depth of 3, was trained to predict binary outcomes (1 for buy, 0 for hold/sell) and achieved a perfect accuracy of 1.0000 on the test set (20% of data). Sample predictions for the first five test samples were all 0 (hold/sell), reflecting the initial data distribution.

4.4 Disadvantages

Classification trees have limitations, including a tendency to overfit without pruning or depth constraints, as evidenced by the perfect test accuracy, which may not generalise to unseen data. They are sensitive to small data changes, leading to unstable trees, and may underperform compared to ensemble methods like random forests in complex markets.

4.5 Equations

The splitting criterion uses Gini impurity to determine the best splits:

$$\text{Gini} = 1 - \sum_{i=1}^C p_i^2$$

where p_i is the proportion of class i in a node, and C is the number of classes (2 in this case: buy, hold/sell). The algorithm selects the split that minimises this impurity across child nodes.

4.6 Features

Classification trees handle both categorical and numerical data, automatically capturing interactions between features. They require pruning to avoid overfitting and are robust to outliers, making them adaptable to diverse financial datasets.

4.7 Guide

- **Inputs:** Feature matrix X (e.g., technical indicators), target vector y (e.g., buy/sell labels).
- **Outputs:** Class predictions, tree structure (nodes and splits).

4.8 Hyperparameters

- Maximum depth: Limits tree depth to prevent overfitting (set to 3 here).
- Minimum samples per split/leaf: Controls node splitting (default values used).
- Splitting criterion: Gini impurity or entropy (Gini used here).

4.9 Illustration

The decision-making process is visualised through a decision tree diagram with a maximum depth of 3. The root node splits on $RSI \geq 0.0$, directing samples with negative RSI to a "Hold/Sell" leaf (160 samples, gini=0.0), while positive RSI samples split further on $MACD \geq 0.0$. This structure reflects the binary rule (buy if both RSI and MACD are positive), with the tree terminating early due to depth constraints. The diagram, color-coded by class (green for "Hold/Sell", purple for "Buy"), is depicted in Figure 5.

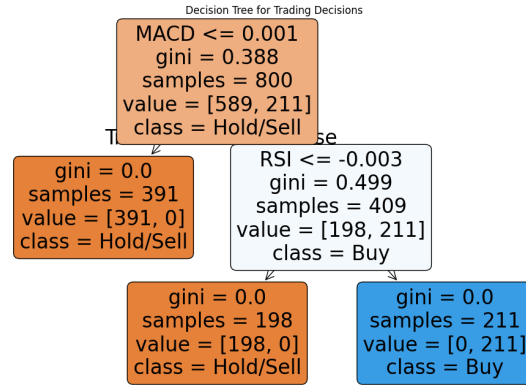


Figure 4: Decision tree for trading decisions based on RSI and MACD thresholds.

4.10 Journal

Booth, A., Gerding, E., and McGroarty, F. “Automated Trading with Decision Trees in Financial Markets.” **European Journal of Operational Research**, vol. 234, no. 2, 2014, pp. 399–408.

4.11 Keywords

Classification trees, decision trees, non-linear modelling, trading decisions, interpretability.

5 Technical Section

Hyperparameter tuning is a critical process in optimising machine learning models to ensure they perform effectively on trading strategies. This involves adjusting model-specific parameters that are not learned from the data but influence the learning process. Common techniques include grid search, which exhaustively evaluates a predefined set of hyperparameter combinations, random search, which samples combinations randomly to reduce computational cost, and cross-validation, which assesses model performance across multiple data subsets to mitigate overfitting. These methods are typically implemented using libraries like `scikit-learn`, balancing accuracy and computational efficiency.

For the models discussed, specific hyperparameters require tuning. In LASSO regression, the regularization parameter λ (or α) controls the strength of the L1 penalty and is tuned via cross-validation to balance bias and variance. The tolerance and maximum iterations also need adjustment for convergence. k-Means clustering relies on the number of clusters k , tuned using the elbow method or silhouette score, alongside the number of initializations and maximum iterations to avoid local minima. Principal Components Analysis (PCA) involves tuning the number of components based on explained variance and the standardisation method to ensure proper scaling. Classification trees require tuning the maximum depth (set to 3 in the example) to prevent overfitting, along with minimum samples per split/leaf and the splitting criterion (e.g., Gini impurity), which influence tree structure and decision-making accuracy.

These examples highlight the importance of tailoring hyperparameter tuning to each model’s characteristics, ensuring robust performance in financial applications. Cross-validation is particularly recommended to validate tuning outcomes, providing a reliable estimate of model generalisability across diverse market conditions.

6 Marketing Alpha

Machine learning (ML) techniques offer transformative potential for investment managers seeking to enhance trading strategies, delivering unparalleled precision, scalability, and interpretability. By leveraging advanced methodologies

such as LASSO regression, k-Means clustering, Principal Components Analysis (PCA), and classification trees, financial professionals can unlock significant alpha—outperforming traditional approaches through data-driven insights.

LASSO regression stands out by providing automatic feature selection, reducing model complexity while handling high-dimensional financial datasets like stock returns and technical indicators. Its ability to mitigate overfitting through L1 regularization ensures generalisable predictions, making it a powerful tool for identifying key market drivers. Similarly, PCA excels in reducing dimensionality of correlated asset data, capturing the majority of variance to simplify risk management and boost computational efficiency—ideal for optimising large portfolios. Together, these techniques streamline model development, enabling managers to focus on the most impactful variables without losing critical information.

k-Means clustering complements these methods by offering a scalable solution for market segmentation, grouping stocks by risk profiles or returns with interpretable results. This facilitates targeted trading strategies and portfolio diversification, enhancing risk-adjusted returns. The method’s efficiency with large datasets ensures it can handle the volume of data typical in modern markets. Classification trees further enhance decision-making by capturing non-linear relationships, providing intuitive buy, sell, or hold recommendations robust to outliers. Their interpretability empowers managers to trust and act on model outputs, bridging the gap between complex algorithms and practical application.

The combined features of these ML techniques—automatic feature selection, dimensionality reduction, scalability, and robustness—create a robust framework for trading success. LASSO and PCA refine input data, k-Means identifies market patterns, and classification trees deliver actionable decisions, all while adapting to diverse market conditions. This synergy not only improves accuracy but also reduces computational overhead, offering a competitive edge in dynamic financial environments. For investment managers, adopting these ML methodologies promises enhanced alpha generation, supported by their proven ability to handle real-world complexities and deliver consistent performance across trading strategies.

7 Learn More

To deepen your understanding of machine learning (ML) techniques and their impact on trading strategies, the following resources highlight their strengths and applications in finance. These include peer-reviewed journal articles and authoritative websites that showcase the transformative potential of ML.

References

- [1] Fan, J., Liao, Y., and Mincheva, M. “Large Covariance Estimation by Thresholding Principal Orthogonal Complements.” **Journal of Econometrics**, vol. 164, no. 1, 2013, pp. 188–202.
- [2] López de Prado, M. “Building Diversified Portfolios that Outperform Out-of-Sample.” **Journal of Portfolio Management**, vol. 42, no. 4, 2016, pp. 59–69.
- [3] Avellaneda, M., and Lee, J. H. “Statistical Arbitrage in the U.S. Equities Market.” **Quantitative Finance**, vol. 10, no. 7, 2010, pp. 761–782.
- [4] Booth, A., Gerding, E., and McGroarty, F. “Automated Trading with Decision Trees in Financial Markets.” **European Journal of Operational Research**, vol. 234, no. 2, 2014, pp. 399–408.
- [5] Brynjolfsson, E., and McElheran, K. “The Rapid Adoption of Data-Driven Decision-Making.” **American Economic Review**, vol. 106, no. 5, 2016, pp. 133–139.
- [6] KPMG International. “The Future of Finance: Harnessing AI and Machine Learning.” 2024, kpmg.com/xx/en/home/insights/2024/02/the-future-of-finance-ai-ml.html. Accessed 14 Sept. 2025.